

# Robust Pupil Detection Using Classical Computer Vision and Deep Learning

Part II: Transition to YOLOv8-Based Detection

B.Tech Project 6th Sem Report

G Madhukar  
EP23BTECH11011



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్  
भारतीय प्रौद्योगिकी संस्थान हैदराबाद  
Indian Institute of Technology Hyderabad

Department of Physics  
Supervisor: Dr. Vandana Sharma

Indian Institute of Technology Hyderabad

# Contents

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Abstract</b>                                         | <b>3</b>  |
| <b>2</b> | <b>Recap of Previous Work and Motivation for Change</b> | <b>4</b>  |
| 2.1      | Summary of the U-Net Approach                           | 4         |
| 2.2      | Limitations of the U-Net Approach                       | 4         |
| <b>3</b> | <b>YOLOv8 Overview</b>                                  | <b>6</b>  |
| 3.1      | Architecture                                            | 6         |
| 3.2      | YOLOv8n: The Nano Variant                               | 7         |
| 3.3      | Training Loop                                           | 7         |
| 3.4      | Forward Pass: Step-by-Step                              | 8         |
| 3.4.1    | Phase 0: Input Tensor                                   | 8         |
| 3.4.2    | Phase 1: The Backbone (Modified CSPDarknet)             | 8         |
| 3.4.3    | Phase 2: The Neck (PAN-FPN)                             | 9         |
| 3.4.4    | Phase 3: The Head (Anchor-Free, Decoupled)              | 10        |
| 3.5      | Model Export for Raspberry Pi Deployment                | 11        |
| <b>4</b> | <b>Dataset Construction for YOLOv8</b>                  | <b>13</b> |
| 4.1      | Motivation for Self-Supervised Label Generation         | 13        |
| 4.2      | Source Data                                             | 13        |
| 4.3      | Bounding Box Generation from Ellipse Parameters         | 13        |
| 4.4      | Dataset Split and Directory Structure                   | 14        |
| <b>5</b> | <b>YOLOv8 Training Setup and Process</b>                | <b>15</b> |
| 5.1      | Training Configuration                                  | 15        |
| 5.2      | Rectangular Training                                    | 16        |
| 5.3      | Transfer Learning and Convergence                       | 16        |
| <b>6</b> | <b>Training Results and Model Quality</b>               | <b>17</b> |
| 6.1      | Loss Convergence                                        | 17        |
| 6.2      | Detection Metrics at Final Epoch                        | 18        |
| 6.3      | F1 Score and Confidence Threshold Robustness            | 19        |
| 6.4      | Confusion Matrix and Localisation Summary               | 20        |
| <b>7</b> | <b>Offline Inference Pipeline</b>                       | <b>22</b> |
| 7.1      | Overview                                                | 22        |
| 7.2      | Input Handling                                          | 22        |
| 7.3      | Detection and Axis Ratio Filter                         | 22        |
| 7.4      | Pupil Area from Bounding Box                            | 23        |
| 7.5      | Outputs                                                 | 23        |
| <b>8</b> | <b>Live Detection System</b>                            | <b>24</b> |
| 8.1      | Overview                                                | 24        |

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| 8.2       | System Architecture . . . . .                     | 24        |
| 8.3       | YoloPupilDetector and Candidate Scoring . . . . . | 24        |
| 8.4       | Optional ROI Masking . . . . .                    | 25        |
| 8.5       | Recording and Data Logging . . . . .              | 25        |
| <b>9</b>  | <b>Results and Discussion</b>                     | <b>27</b> |
| 9.1       | Quantitative Evaluation . . . . .                 | 27        |
| 9.2       | Comparison with U-Net . . . . .                   | 27        |
| 9.3       | Limitations . . . . .                             | 28        |
| <b>10</b> | <b>Conclusion and Future Work</b>                 | <b>29</b> |
| 10.1      | Conclusion . . . . .                              | 29        |
| 10.2      | Future Work . . . . .                             | 29        |
|           | <b>Declaration</b>                                | <b>31</b> |
|           | <b>Acknowledgements</b>                           | <b>32</b> |
|           | <b>List of Abbreviations</b>                      | <b>34</b> |

# Chapter 1

## Abstract

In the previous semester’s work (5th Semester Report), a hybrid system was developed that combined a classical image processing pipeline with a U-Net deep learning model to address the problem of robust pupil detection in noisy eye videos. The classical pipeline, based on Canny edge detection, contour analysis, and ellipse fitting, frequently failed on frames containing eyelashes, corneal reflections, or dark moles near the pupil boundary — producing frames where the recorded pupil area was zero.

The U-Net model was trained to reconstruct ellipse-fitted images from raw eye frames, allowing the classical detector to re-run on the AI-generated outputs and recover pupil areas in previously failed frames. While this approach showed significant improvement across three datasets (D1, D2, D3), it revealed a new set of practical limitations related to training complexity, inference speed, and the quality of recovered ellipses.

This report documents the 6th Semester work, which focuses on replacing the U-Net with a **YOLOv8n**-based object detection model. YOLOv8n (*nano* variant of YOLO version 8) is a lightweight, single-stage object detector that directly predicts bounding boxes around the pupil region, bypassing the need for pixel-level image reconstruction. This approach achieves faster training convergence, lower inference latency, and improved detection accuracy, especially under challenging occlusion and noise conditions.

Beyond offline processing of pre-recorded videos and frame folders, this semester also explored **live real-time pupil detection** directly from the Raspberry Pi camera stream. A dedicated GUI-based application was developed for this purpose, using the NCNN format of the trained YOLOv8n model for optimized on-device inference.

The overall system goal remains the same: to reliably estimate pupil area frame by frame from eye videos captured using a Raspberry Pi-based setup, supporting downstream fatigue classification through analysis of the rate of change of pupil area over time.

# Chapter 2

## Recap of Previous Work and Motivation for Change

### 2.1 Summary of the U-Net Approach

The 5th Semester approach can be summarized as follows:

1. The classical pipeline processes each frame and categorizes it into either `area_detected_frames/` (successful) or `zero_area_frames/` (failed).
2. A U-Net is trained using pairs of (original frame, ellipse-fitted frame) derived from the successfully detected frames.
3. For zero-area frames, the trained U-Net generates a predicted ellipse-fitted image.
4. The classical detector is re-run on these AI-generated images to extract a valid pupil area.

The method achieved strong quantitative results:

Table 2.1: U-Net performance recap across datasets.

| Dataset | Total Frames | Zero-Area Before | Zero-Area After | Reduction (%) |
|---------|--------------|------------------|-----------------|---------------|
| D1      | 729          | 24               | 0               | 100.00%       |
| D2      | 4668         | 270              | 19              | 92.96%        |
| D3      | 8624         | 6211             | 358             | 94.24%        |

### 2.2 Limitations of the U-Net Approach

Despite the improvement in detection, the U-Net approach exposed several practical limitations that motivated the transition to YOLOv8:

- **Slow training convergence.** The U-Net, being a full image-to-image translation network, requires many epochs to converge on even a modest dataset, and is not well-suited for rapid iteration.
- **Pixel-level prediction overhead.** The U-Net outputs a full-resolution image rather than a compact representation of the pupil location. This is computationally wasteful since the downstream task only requires the ellipse parameters (center, axes, angle).

- **Inherited classical bias.** Since the U-Net is trained to mimic the classical detector's ellipse outputs, any systematic error or geometric inaccuracy in the classical fitting is directly learned and reproduced by the network.
- **Two-stage inference.** Recovering a pupil area from a zero-area frame required two steps: (1) U-Net generates the ellipse image, then (2) the classical detector re-runs on that image. This introduced additional failure points and computational overhead.
- **Limited spatial accuracy.** The U-Net-predicted ellipses were occasionally blurry or slightly misaligned with the true pupil boundary, reducing the reliability of the recovered area.
- **Not suitable for real-time deployment.** The two-stage architecture and the computational cost of running a full encoder-decoder network make the U-Net unsuitable for live inference on the Raspberry Pi.

These observations motivated the adoption of YOLOv8n, a direct detection model that locates the pupil using a bounding box and provides a clean, interpretable output in a single forward pass — and that can be exported to efficient formats for embedded deployment.

# Chapter 3

## YOLOv8 Overview

### 3.1 Architecture

YOLO (*You Only Look Once*) is a family of real-time object detection models that frames detection as a single regression problem: the network directly predicts bounding box coordinates and class probabilities from the full image in one pass [8]. Most older detection frameworks used a two-stage approach — first proposing candidate regions that might contain an object, then classifying each region separately. YOLO eliminates this two-step process entirely, passing the image through the network exactly once and producing the object’s location and class label simultaneously. This makes YOLO significantly faster while remaining highly accurate.

YOLOv8, developed by Ultralytics, is the eighth major generation of this family and introduces several architectural improvements over its predecessors [7].

The YOLOv8 architecture consists of three main components, each with a distinct responsibility:

- **Backbone (CSPDarknet — Cross Stage Partial Darknet).** This is the “eye” of the network. It scans the raw input image layer by layer, building up an increasingly abstract description. Early layers detect simple things like edges and brightness gradients; deeper layers detect shapes, textures, and object-level patterns such as the circular, dark appearance of a pupil. The backbone uses a sequence of convolutional blocks with residual (skip) connections to extract these multi-scale features efficiently.
- **Neck (PAN-FPN — Path Aggregation Network with Feature Pyramid Network).** After the backbone runs, small objects can become difficult to localise in the deep feature maps because spatial detail has been compressed. The neck solves this by fusing features across scales: it passes rich semantic information downward (FPN — Feature Pyramid Network) and sharp spatial information upward (PAN — Path Aggregation Network), so that every level of the network simultaneously knows *what* it is looking at and *exactly where* it is.
- **Head.** The head reads the neck’s fused feature maps and produces the final output. YOLOv8 uses a **decoupled, anchor-free** head that separately predicts bounding box coordinates and classification scores. “Anchor-free” means the network does not rely on pre-defined reference box shapes; instead, for each point in the feature map it directly predicts the distance to the four edges of the object, removing the need to manually tune anchor parameters for each new dataset.

## 3.2 YOLOv8n: The Nano Variant

The **n** (nano) variant of YOLOv8 is the smallest and fastest model in the YOLOv8 family, which comes in five sizes: **n** (nano), **s** (small), **m** (medium), **l** (large), and **x** (extra-large). Larger variants achieve higher accuracy on complex multi-class benchmarks but require more memory and computation. Since this project detects a single, visually distinctive class (**pupil**) on a Raspberry Pi with limited processing power, the nano variant strikes the best balance between speed and accuracy.

Table 3.1: YOLOv8n model characteristics.

| Property          | Value                                                         |
|-------------------|---------------------------------------------------------------|
| Parameters        | ~3.2 million                                                  |
| Input resolution  | $640 \times 640$ (used in this project)                       |
| Detection type    | Anchor-free, single-stage                                     |
| Output            | Bounding box ( $x_c$ , $y_c$ , $w$ , $h$ ) + confidence score |
| Deployment format | NCNN (ARM-optimised)                                          |

Here  $x_c$  is the horizontal pixel coordinate of the detected box centre (i.e. how far from the left edge of the frame the pupil centre is),  $y_c$  is the vertical pixel coordinate of the box centre (how far from the top),  $w$  is the box width in pixels, and  $h$  is the box height in pixels.

For this project, the model is trained to detect a single class: **pupil**. The predicted bounding box is then used to derive the pupil area as described in Chapter 7.

## 3.3 Training Loop

Before describing the architecture in detail, it is helpful to understand how the network learns. Training is an iterative process that repeats the following four steps for every mini-batch of images:

1. **Forward pass.** The image flows from the input through the Backbone, Neck, and Head. In early training, the network's output is essentially random — it has not yet learned to find pupils.
2. **Compute loss.** The predicted bounding box is compared to the ground-truth label. The total loss combines three terms: how wrong the box position and size are (box loss), how uncertain the class prediction is (classification loss — Cls loss), and how accurate the edge distribution predictions are (DFL loss). Larger losses mean larger errors.
3. **Backpropagation.** The error signal is traced back through every layer of the network using the chain rule of calculus. This computes, for each of the 3.2 million trainable weights, how much that weight contributed to the total loss and in which direction it should be adjusted to reduce the error.
4. **Weight update.** Each weight is nudged slightly in the direction that reduces the loss. This project uses the **Adam optimiser**, which adapts the step size for each

weight individually, making convergence faster and more stable.

After thousands of such iterations across 50 epochs (one epoch = one complete pass through the entire training dataset), the weights converge to values that allow the network to reliably draw a tight bounding box around the pupil on every frame.

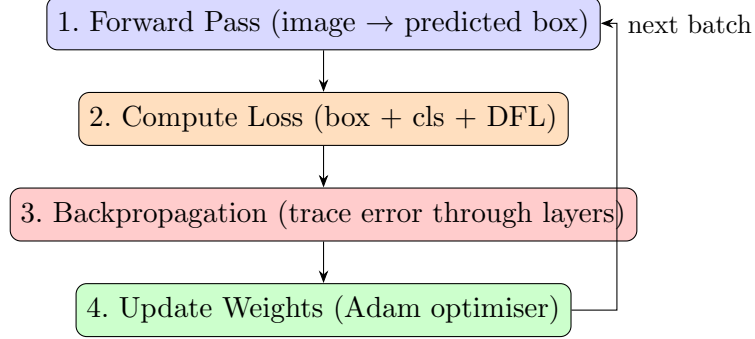


Figure 3.1: One training iteration. The cycle repeats for every mini-batch across all 50 epochs. The feedback arrow on the right represents the continuous loop: after each weight update the next batch is fed forward immediately.

## 3.4 Forward Pass: Step-by-Step

Understanding the YOLOv8 forward pass requires tracing how raw pixel values are progressively transformed into bounding box predictions. The process is divided into three phases corresponding to the Backbone, Neck, and Head.

### 3.4.1 Phase 0: Input Tensor

Before the forward pass begins, the input image is resized and normalized. The resulting input tensor is:

$$X_{\text{in}} \in \mathbb{R}^{H \times W \times 3} \quad (3.1)$$

where  $H$  and  $W$  are the image height and width respectively, and 3 denotes the three colour channels (R, G, B). For this project,  $H = W = 640$ .

### 3.4.2 Phase 1: The Backbone (Modified CSPDarknet)

The backbone's role is to extract increasingly abstract visual features from the input. It does this by progressively halving the spatial dimensions ( $H$ ,  $W$ ) while increasing the number of feature channels, so that early layers capture low-level features (edges, textures) and deeper layers capture high-level semantics (shapes, object parts).

#### Spatial Reduction via Strided Convolutions

The tensor is passed through convolutional layers with stride 2. Each such layer halves both  $H$  and  $W$  while doubling the number of channels. The standard convolution operation at position  $(i, j)$  for output channel  $k$  is:

$$Z_{i,j}^{(k)} = \sum_c \sum_p \sum_q W_{c,p,q}^{(k)} \cdot X_{c,2i+p,2j+q} + b^{(k)} \quad (3.2)$$

where  $W^{(k)}$  are the learned kernel weights,  $b^{(k)}$  is a bias, and the stride of 2 in the indexing  $(2i, 2j)$  is what produces the spatial downsampling.

### The C2f Module (CSP Logic)

YOLOv8 replaces the older C3 module with the **C2f** (Cross Stage Partial bottleneck with 2 convolutions) module. This module addresses the vanishing gradient problem by splitting the feature tensor into two halves before processing:

$$[x_1, x_2] = \text{Split}(X_{\text{conv}}) \quad (3.3)$$

Here,  $x_1$  acts as a *memory path* that bypasses the heavy computation and preserves gradients for earlier layers;  $x_2$  passes through a chain of bottleneck blocks that learn progressively refined features. Let the bottleneck outputs be  $y_1, y_2, \dots, y_n$ . The C2f output is formed by concatenating all paths and applying a final convolution:

$$X_{\text{out}} = \text{Conv}(\text{Concat}(x_1, x_2, y_1, y_2, \dots, y_n)) \quad (3.4)$$

This design means that each gradient update can flow directly back through  $x_1$  without passing through the bottleneck stack, significantly improving training stability on small datasets such as the one used in this project.

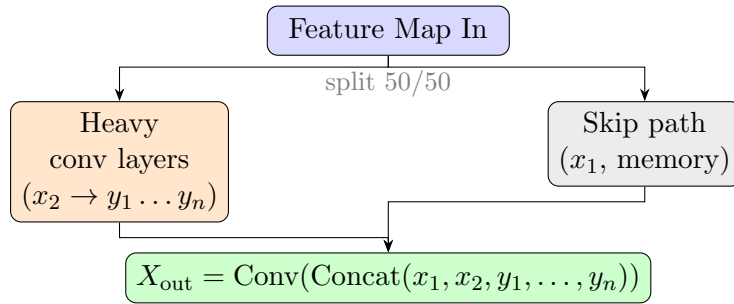


Figure 3.2: C2f module: the feature map is split 50/50. One half ( $x_1$ ) bypasses the heavy layers entirely (memory/skip path); the other half ( $x_2$ ) passes through bottleneck blocks. All paths are concatenated and refined by a final convolution.

### Multi-Scale Backbone Outputs

The backbone does not output a single tensor. Instead, it taps the network at three depths to produce feature maps at three spatial scales:

- $P_3$  — stride 8, **high spatial resolution**, weak semantics (good for localising small or fine-grained details).
- $P_4$  — stride 16, **medium resolution**, medium semantics.
- $P_5$  — stride 32, **low spatial resolution**, strong semantics (good for recognising what the object is, even if location is coarse).

#### 3.4.3 Phase 2: The Neck (PAN-FPN)

The neck's role is to fuse the three backbone feature maps so that every scale benefits from both the semantic strength of deep features and the spatial precision of shallow features. YOLOv8 uses a **Path Aggregation Network (PAN)** combined with a **Feature Pyramid Network (FPN)**.

### Top-Down Pathway (FPN)

The FPN injects semantic information *downward* from deep to shallow layers.  $P_5$  is upsampled (spatial dimensions doubled) and concatenated with  $P_4$ , then refined by a C2f block:

$$F_4 = \text{C2f}(\text{Concat}(P_4, \text{Upsample}(P_5))) \quad (3.5)$$

This is repeated one level further to enrich  $P_3$ :

$$F_3 = \text{C2f}(\text{Concat}(P_3, \text{Upsample}(F_4))) \quad (3.6)$$

After the FPN pass,  $F_3$  and  $F_4$  carry semantic context from the deep layers while retaining the fine spatial detail of the shallower layers.

### Bottom-Up Pathway (PAN)

The PAN then injects localization information *upward* from shallow to deep layers.  $F_3$  is downsampled (spatial dimensions halved) and fused back with  $F_4$ :

$$N_4 = \text{C2f}(\text{Concat}(F_4, \text{Downsample}(F_3))) \quad (3.7)$$

This is repeated to produce the final high-level enriched map:

$$N_5 = \text{C2f}(\text{Concat}(P_5, \text{Downsample}(N_4))) \quad (3.8)$$

The three neck outputs  $N_3 = F_3$ ,  $N_4$ , and  $N_5$  each contain a rich mixture of “what” (semantic) and “where” (spatial) information, making them well-suited for accurate pupil localization at multiple scales.

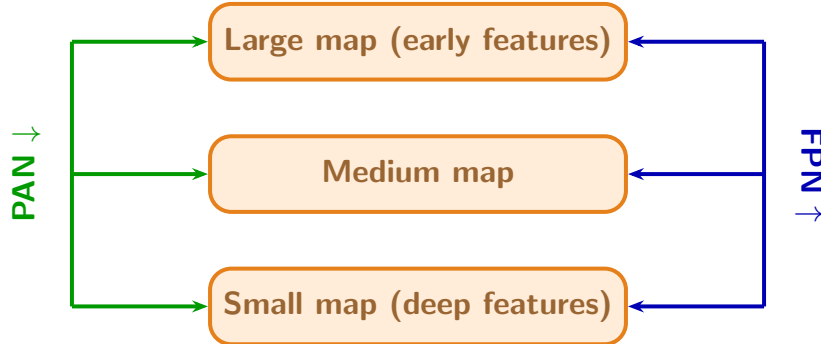


Figure 3.3: PAN-FPN neck. The FPN pathway (blue, right) propagates semantic information from  $P_3$  downward through all scales. The PAN pathway (green, left) propagates sharp spatial detail from  $P_5$  upward. After both passes, every scale carries both strong semantic understanding and precise localization.

### 3.4.4 Phase 3: The Head (Anchor-Free, Decoupled)

The head converts the enriched feature maps from the neck into final detection outputs. YOLOv8 uses a **decoupled head**: for each of the three feature maps ( $N_3$ ,  $N_4$ ,  $N_5$ ), the data stream is split into two independent branches that specialise in different sub-tasks.

### Classification Branch

This branch predicts the probability that the object in each grid cell belongs to each class. For a feature map of spatial size  $H' \times W'$  and  $C$  object classes, the output tensor is:

$$\text{Output}_{\text{cls}} \in \mathbb{R}^{C \times H' \times W'} \quad (3.9)$$

Since this project uses only one class (**pupil**),  $C = 1$ , so this branch produces a single confidence map per scale.

### Bounding Box Regression Branch

Rather than directly regressing four absolute coordinates, YOLOv8 uses **Distribution Focal Loss (DFL)** logic. For each of the four box edges (left, top, right, bottom), the network outputs a probability distribution over 16 discrete “bins”. The expected value of this distribution gives the predicted edge distance. The output tensor size per scale is:

$$\text{Output}_{\text{reg}} \in \mathbb{R}^{(4 \times 16) \times H' \times W'} \quad (3.10)$$

Using a distribution instead of a point estimate makes the model better calibrated: when the pupil boundary is ambiguous (e.g. partially occluded by an eyelash), the distribution becomes broad rather than confidently wrong.

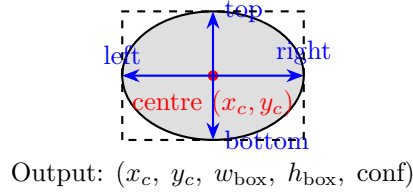


Figure 3.4: Anchor-free head prediction. For each candidate grid point, the head predicts the distance from that point to each of the four bounding box edges (left, right, top, bottom). No pre-defined anchor shapes are required. The dashed rectangle is the resulting bounding box, and  $(x_c, y_c)$  is the detected pupil centre.

### Final Output Assembly

The head flattens and concatenates the tensors from all three scales into a single prediction array. During inference this array is passed through **Non-Maximum Suppression (NMS)**, which retains the highest-scoring non-overlapping box, yielding the final pupil bounding box  $(x_c, y_c, w, h)$  and confidence score  $c \in [0, 1]$ .

## 3.5 Model Export for Raspberry Pi Deployment

The YOLOv8n model is trained in PyTorch (**.pt** format) on a PC with a GPU. Since running a full PyTorch model directly on the Raspberry Pi is slow, the trained model is **exported to NCNN format** — a high-performance neural network inference framework optimized for ARM-based embedded processors [?].

The export produces a folder (**best\_ncnn\_model/**) containing the NCNN parameter and binary weight files. Both the offline detection script and the live GUI application load this

NCNN model directly via the Ultralytics API, which transparently handles the NCNN backend. The offline script additionally supports TFLite (`.tflite`) format for further deployment flexibility.

# Chapter 4

## Dataset Construction for YOLOv8

### 4.1 Motivation for Self-Supervised Label Generation

A key constraint of this project is the absence of large manually annotated datasets. Manual annotation of pupil bounding boxes across thousands of video frames is time-intensive and impractical for a single researcher. To overcome this, the same automatic data generation strategy introduced in the 5th Semester work was extended and adapted for YOLOv8.

### 4.2 Source Data

The input data consists of frames categorized by the modified classical detection script into two groups:

- `area_detected_frames/` — frames where the classical pipeline successfully detected the pupil and fitted an ellipse.
- `ellipse_fitted_frames/` — the same frames with the fitted ellipse drawn using `cv2.ellipse`.

### 4.3 Bounding Box Generation from Ellipse Parameters

For each frame in `area_detected_frames/`, the classical pipeline already computes the fitted ellipse parameters. These four parameters fully describe the shape and position of the detected pupil:

- $(x_c, y_c)$  — the **centre of the ellipse**:  $x_c$  is the horizontal pixel coordinate of the pupil centre measured from the left edge of the frame, and  $y_c$  is the vertical pixel coordinate measured from the top edge.
- **MA (Major Axis)** — the length of the *longer* diameter of the fitted ellipse, in pixels. For a roughly circular pupil this corresponds approximately to the width of the pupil.
- **ma (minor axis)** — the length of the *shorter* diameter of the fitted ellipse, in pixels. For a roughly circular pupil this corresponds approximately to the height of the pupil.

A bounding box label is derived directly from these parameters by constructing the tightest axis-aligned rectangle that encloses the ellipse. The rectangle extends half the major axis to the left and right of the centre, and half the minor axis above and below it:

$$x_{\min} = x_c - \frac{MA}{2}, \quad x_{\max} = x_c + \frac{MA}{2} \quad (4.1)$$

$$y_{\min} = y_c - \frac{ma}{2}, \quad y_{\max} = y_c + \frac{ma}{2} \quad (4.2)$$

Here  $x_{\min}$  and  $x_{\max}$  are the left and right edges of the bounding box in pixels, and  $y_{\min}$  and  $y_{\max}$  are the top and bottom edges. These are then converted to the YOLO annotation format, which requires all coordinates to be *normalized* — divided by the image dimensions — so that labels remain valid across frames of different resolutions (image width  $W$ , image height  $H$ ):

$$x_{\text{norm}} = \frac{x_c}{W}, \quad y_{\text{norm}} = \frac{y_c}{H}, \quad w_{\text{norm}} = \frac{MA}{W}, \quad h_{\text{norm}} = \frac{ma}{H} \quad (4.3)$$

All four values lie in  $[0, 1]$ : a centre at the middle of the frame gives  $x_{\text{norm}} = 0.5$ , a box spanning the full frame width gives  $w_{\text{norm}} = 1.0$ , and so on. Each annotation is saved as a `.txt` file containing one line:

```
<class_id>  x_norm  y_norm  w_norm  h_norm
```

with `class_id = 0` for the single `pupil` class.

## 4.4 Dataset Split and Directory Structure

The labeled dataset is split into training and validation subsets using a standard 80/20 ratio. The directory structure follows the YOLO convention required by Ultralytics:

```
dataset/
  images/
    train/    (80% of frames)
    val/      (20% of frames)
  labels/
    train/    (corresponding .txt annotation files)
    val/
```

A `data.yaml` configuration file specifies the paths and class names:

```
train: dataset/images/train
val:   dataset/images/val
nc: 1
names: ['pupil']
```

This pipeline requires no manual annotation: labels are fully derived from the classical detector's successful outputs, maintaining the self-supervised data generation philosophy established in the previous semester.

# Chapter 5

## YOLOv8 Training Setup and Process

### 5.1 Training Configuration

The YOLOv8n model was trained on a PC with a CUDA-capable GPU using the Ultralytics Python API. The training script used is as follows:

```
from ultralytics import YOLO

def train():
    model = YOLO("yolov8n.pt")
    model.train(
        data=r"D:\Madhukar\YOLO\YAML\data_1.yaml",
        epochs=50,
        imgsz=640,
        rect=True,
        save_period=10,
        device='cuda'
    )

if __name__ == "__main__":
    train()
```

The key hyperparameters are summarized in Table 5.1.

Table 5.1: YOLOv8n training configuration.

| Hyperparameter                       | Value                        |
|--------------------------------------|------------------------------|
| Base model                           | yolov8n.pt (COCO pretrained) |
| Number of epochs                     | 50                           |
| Image size                           | $640 \times 640$             |
| Rectangular training ( <b>rect</b> ) | Enabled                      |
| Checkpoint save period               | Every 10 epochs              |
| Device                               | CUDA (GPU)                   |
| Number of classes                    | 1 ( <b>pupil</b> )           |
| Confidence threshold (inference)     | 0.25                         |

## 5.2 Rectangular Training

The `rect=True` flag enables **rectangular training**, where images within each batch are padded to the same aspect ratio rather than to a fixed square. This reduces the amount of padding added to non-square images such as the  $640 \times 480$  eye frames captured by the PiCamera, which reduces wasted computation on padded regions and improves both training efficiency and detection accuracy on non-square inputs.

## 5.3 Transfer Learning and Convergence

Rather than training from random initialization, the model is fine-tuned starting from COCO-pretrained weights (`yolov8n.pt`). **COCO** (Common Objects in Context) is a large-scale benchmark dataset containing over 330,000 images spanning 80 object categories. A model pre-trained on COCO has already spent hundreds of GPU-hours learning to detect edges, textures, blob-like regions, and complex shapes — precisely the kind of low-level features that a dark circular pupil also exhibits.

A useful analogy: instead of teaching a student everything about anatomy from scratch, we start with a student who already knows basic biology and only teach them the specific topic of ophthalmology. The foundation is already there; the remaining learning is fast and targeted.

Transfer learning offers two major practical advantages:

- The backbone has already learned general low-level visual features (edges, textures, blob-like structures) that transfer directly to pupil detection.
- Fine-tuning on the pupil dataset requires far fewer gradient steps to converge compared to training from scratch, keeping the total training time modest even at 50 epochs.

Checkpoints are saved every 10 epochs (`save_period=10`), allowing recovery from interruptions and enabling inspection of intermediate model quality. After training, the best weights (based on validation metrics) are saved as `best.pt` and subsequently exported to NCNN format for Raspberry Pi deployment.

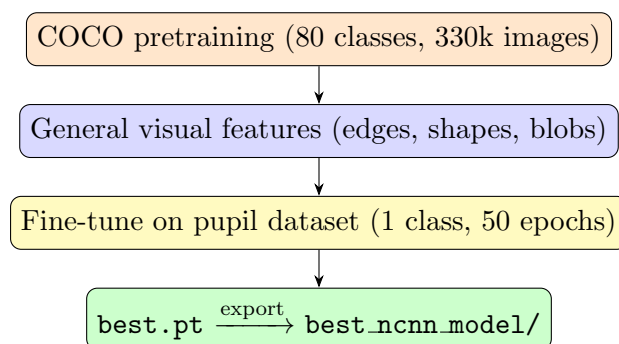


Figure 5.1: Transfer learning pipeline. The model starts from COCO-pretrained weights (general visual knowledge) and is fine-tuned on the self-supervised pupil dataset. The best checkpoint is then exported to NCNN format for deployment on the Raspberry Pi.

# Chapter 6

## Training Results and Model Quality

### 6.1 Loss Convergence

All three training losses — box loss, classification loss, and DFL loss — decrease monotonically over the 50 training epochs without divergence between the training and validation curves. Table 6.1 reports the final epoch values.

Table 6.1: Final epoch loss values (epoch 50).

| Loss     | Train  | Val    |
|----------|--------|--------|
| Box loss | 0.1865 | 0.1469 |
| Cls loss | 0.1533 | 0.1340 |
| DFL loss | 0.7552 | 0.7471 |

The three loss terms capture different aspects of the detection task:

- **Box loss** measures how accurately the predicted bounding box edges match the ground-truth label. Lower values indicate that the network has learned to tightly enclose the pupil region.
- **Classification loss** measures how confidently the network assigns the detected region to the `pupil` class. The large initial drop (from 2.04 to 0.15 by epoch 50) reflects the model rapidly learning to distinguish the dark pupil region from iris, eyelid, and background.
- **DFL loss** (Distribution Focal Loss) penalizes errors in the edge distribution prediction described in Section 3.4. Its near-equal train and validation values (0.755 vs. 0.747) confirm that the distribution predictions generalize well to unseen frames.

Crucially, all validation losses track their training counterparts without diverging, which confirms that **no overfitting** occurred. This is attributed to the use of transfer learning from COCO-pretrained weights, which provides a well-regularised starting point.

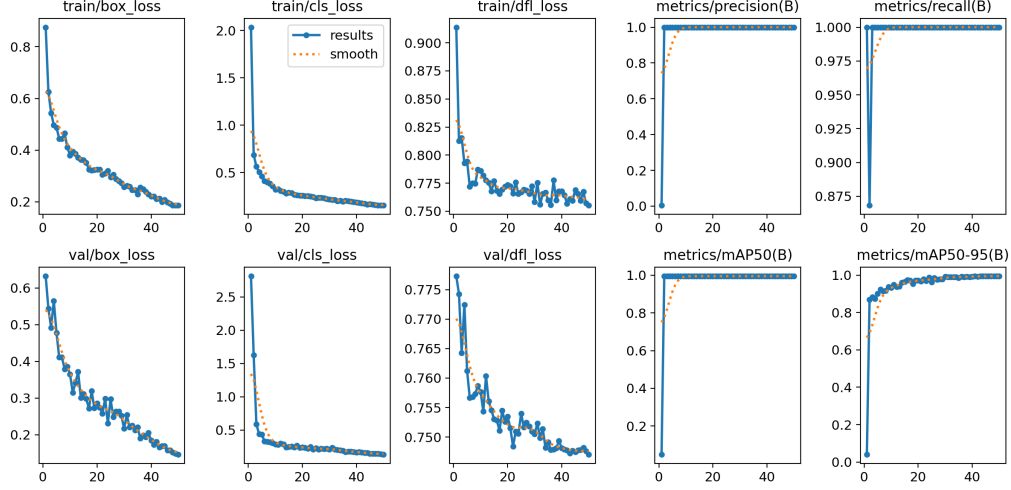


Figure 6.1: Training and validation loss curves across 50 epochs. All curves decrease smoothly and converge without divergence, indicating stable learning.

## 6.2 Detection Metrics at Final Epoch

Table 6.2 reports the standard object detection metrics evaluated on the validation set at the final epoch.

Table 6.2: Final validation metrics (epoch 50).

| Metric          | Value  |
|-----------------|--------|
| Precision       | 0.9999 |
| Recall          | 1.0000 |
| mAP@0.5         | 0.9950 |
| mAP@0.5:0.95    | 0.9949 |
| F1 score (peak) | 1.0000 |

Each metric is defined and interpreted as follows:

- **Precision** is the fraction of all detections that are correct:

$$\text{Precision} = \frac{TP}{TP + FP} \approx 0.9999 \quad (6.1)$$

where  $TP$  is the number of true positive detections and  $FP$  is the number of false positives (spurious detections). A value near 1 means the model almost never fires a detection in the wrong location.

- **Recall** is the fraction of all ground-truth pupils that are found:

$$\text{Recall} = \frac{TP}{TP + FN} = 1.0000 \quad (6.2)$$

where  $FN$  is the number of missed detections. A recall of 1 means that *every* labelled pupil in the validation set was successfully detected.

- **mAP@0.5** is the mean Average Precision at an Intersection over Union (IoU) threshold of 0.5. The IoU between a predicted box  $\hat{B}$  and the ground-truth box  $B$  is:

$$\text{IoU}(\hat{B}, B) = \frac{|\hat{B} \cap B|}{|\hat{B} \cup B|} \quad (6.3)$$

A detection is counted as correct only if  $\text{IoU} \geq 0.5$ , i.e. the predicted box overlaps the ground-truth box by at least 50%. The mAP@0.5 score of 0.995 indicates near-perfect localization at this standard threshold.

- **mAP@0.5:0.95** averages the AP over ten IoU thresholds from 0.50 to 0.95 in steps of 0.05:

$$\text{mAP}_{0.5:0.95} = \frac{1}{10} \sum_{k=0}^9 \text{AP}(0.50 + 0.05k) = 0.9949 \quad (6.4)$$

This is a far stricter metric than mAP@0.5, as it requires the predicted box to overlap the ground truth by up to 95%. The score of 0.9949 confirms that the predicted bounding boxes are geometrically very close to the annotated ground truth on nearly all validation frames.

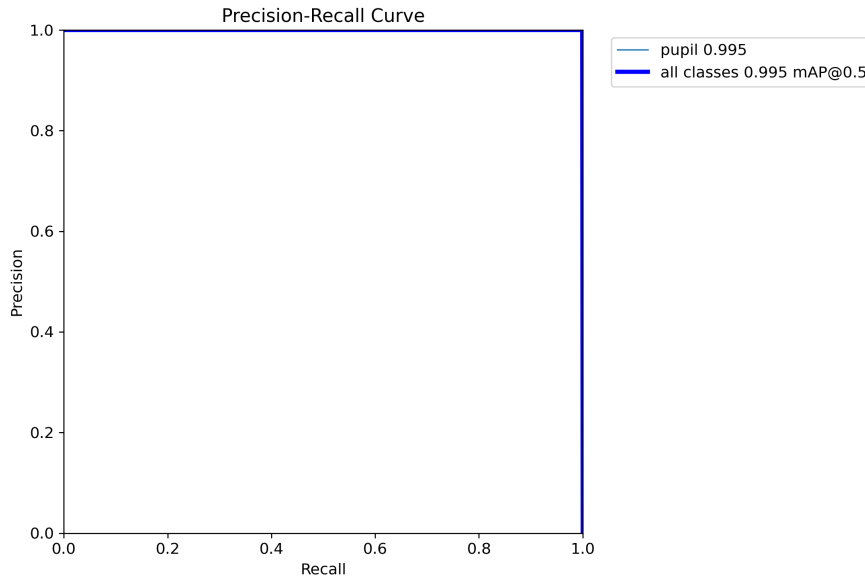


Figure 6.2: Precision–Recall curve on the validation set. The area under the curve (mAP@0.5 = 0.995) is very close to 1, confirming strong detection performance across the full range of confidence thresholds.

### 6.3 F1 Score and Confidence Threshold Robustness

The F1 score is the harmonic mean of Precision and Recall:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6.5)$$

It provides a single balanced measure of detection quality, penalizing models that achieve high precision at the cost of recall (or vice versa). The peak  $F_1 = 1.00$  is achieved at a confidence threshold of  $\tau = 0.884$ .

A key practical observation is that the F1 curve remains **flat at 1.0** from  $\tau \approx 0$  all the way to  $\tau \approx 0.88$ . The inference threshold of  $\tau = 0.25$  used in deployment sits well inside this plateau. This means that the system is **not sensitive to the choice of threshold**: a wide range of  $\tau$  values yields identical detection quality, which is important for robust deployment across varying lighting conditions and subjects.

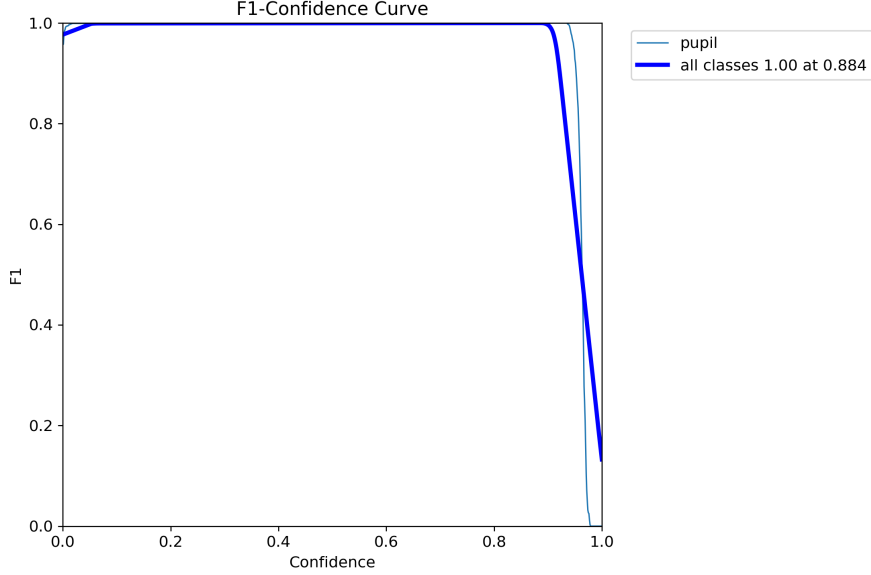


Figure 6.3: F1–Confidence curve. The curve is flat at  $F_1 = 1.00$  over the range  $\tau \in [0, 0.88]$ , demonstrating that the deployed threshold  $\tau = 0.25$  is robust and far from the sensitivity cliff.

## 6.4 Confusion Matrix and Localisation Summary

For a single-class detector, the normalised confusion matrix takes the form:

$$\mathbf{C}_{\text{norm}} = \begin{pmatrix} 1.00 & 0 \\ 0 & - \end{pmatrix} \quad (6.6)$$

where rows and columns correspond to the {pupil, background} classes. The top-left entry of 1.00 indicates a **100% true-positive rate**: every labelled pupil in the validation set was detected with the correct class label. The zero false-positive rate (top-right entry) confirms that no background region was mistakenly classified as a pupil.

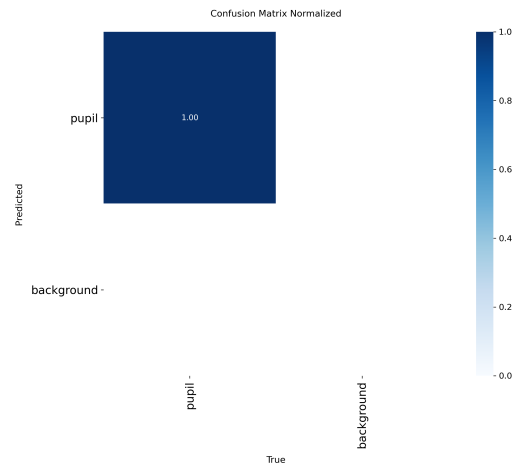


Figure 6.4: Normalised confusion matrix on the validation set. The 100% true-positive rate confirms that no ground-truth pupil was missed.

Taken together, the  $\text{mAP}@0.5:0.95$  score of 0.9949 and the confusion matrix indicate that the predicted bounding boxes overlap the ground truth by at least 95% on nearly all validation frames, providing both accurate localization and correct classification.

# Chapter 7

## Offline Inference Pipeline

### 7.1 Overview

The offline inference script (`code_17_ncnn_v3.py`) processes either a **video file** or a **folder of image frames** and produces pupil area estimates, an annotated output video, and a fatigue classification plot. It supports three model formats transparently — PyTorch (`.pt`), NCNN (folder), and TFLite (`.tflite`) — selectable via a single configuration variable (`MODEL_FORMAT`).

### 7.2 Input Handling

A unified `frame_iterator()` generator function handles both input types:

- **Video file** — frames are decoded sequentially using `cv2.VideoCapture`; the frame rate is read directly from the file metadata.
- **Image folder** — all images are sorted alphabetically and read with `cv2.imread`; the configured `FRAME_RATE` (30 fps) is used.

Both input types yield `(frame_index, frame_bgr, fps)` tuples, making the rest of the pipeline agnostic to the source type. Multiple input sources can be processed in a single run by listing them in `INPUT_SOURCES`.

### 7.3 Detection and Axis Ratio Filter

For each frame, the NCNN model is queried with confidence threshold  $\tau = 0.25$  (meaning only detections where the model is at least 25% confident are considered). Each candidate bounding box then passes through an additional **axis ratio filter** to reject geometrically implausible detections:

$$R_{\text{axis}} = \frac{\min(w_{\text{box}}, h_{\text{box}})}{\max(w_{\text{box}}, h_{\text{box}})} \quad (7.1)$$

Here  $w_{\text{box}}$  is the width of the detected bounding box in pixels and  $h_{\text{box}}$  is its height. The ratio  $R_{\text{axis}}$  measures how circular the box is: a perfect square (corresponding to a perfectly circular pupil) gives  $R_{\text{axis}} = 1$ , while a highly elongated rectangle gives  $R_{\text{axis}} \rightarrow 0$ . Only detections satisfying  $0.60 \leq R_{\text{axis}} \leq 1.00$  are accepted. This rejects spurious elongated detections caused by eyelids or corneal reflections while allowing naturally elliptical pupils. Detections that fail this filter are logged and the frame is recorded as a zero-area frame.

## 7.4 Pupil Area from Bounding Box

Given an accepted bounding box with pixel-space width  $w_{\text{box}}$  (the horizontal span of the box, in pixels) and height  $h_{\text{box}}$  (the vertical span, in pixels), the pupil area is estimated by treating the bounding box as an enclosing ellipse. If we call  $a = w_{\text{box}}/2$  the semi-major axis (half the width) and  $b = h_{\text{box}}/2$  the semi-minor axis (half the height), the area of the ellipse with those semi-axes is  $\pi ab$ , which gives:

$$A_{\text{pupil}} = \frac{\pi}{4} \cdot w_{\text{box}} \cdot h_{\text{box}} \quad (7.2)$$

This is consistent with the classical pipeline's formula  $A = \frac{\pi}{4} \cdot MA \cdot ma$ , where  $MA$  (Major Axis — the longer diameter of the fitted ellipse) and  $ma$  (minor axis — the shorter diameter) are returned directly by `cv2.fitEllipse`. Both expressions compute the area of the inscribed ellipse of the detected bounding box.

## 7.5 Outputs

For each processed source, the script creates a timestamped workspace folder containing:

- `area.txt` — frame-level area, frame index, and elapsed time.
- `zero_area_frames.csv` — frames where no valid pupil was detected.
- `area_detected_frames.csv` — frames with successfully detected areas.
- `recorded.mp4` — annotated output video with ellipse overlays and confidence scores.
- `slope_values.txt` — instantaneous slope of the smoothed area curve.
- `area_plot.png` — fatigue classification plot.
- `log_ellipse_fitting.log` — detailed processing log including skipped frames and axis ratio rejection messages.

# Chapter 8

## Live Detection System

### 8.1 Overview

In addition to offline processing, a **live real-time pupil detection and fatigue analysis system** was developed for the Raspberry Pi (`gui2.py`). This system captures frames directly from the PiCamera2 in real time, runs YOLO inference on each frame, displays the annotated feed in a touchscreen GUI, records a 12-second session on demand, and produces a fatigue classification result at the end of the session.

### 8.2 System Architecture

The live system is built around three components:

- **YoloPupilDetector class** — wraps the NCNN YOLOv8n model (`best_ncnn_model/`) and implements candidate scoring and spatial proximity filtering.
- **Worker thread** — runs the PiCamera2 capture and YOLO inference loop independently of the GUI event loop, pushing display frames to the main thread via `root.after(0, ...)` for thread safety.
- **CustomTkinter GUI** — displays the live annotated camera feed at  $640 \times 240$  resolution, provides Record/Stop/Exit buttons, and renders the area-vs-time plot and fatigue result after the session ends.

### 8.3 YoloPupilDetector and Candidate Scoring

The `detect_best()` method applies the same axis ratio filter as the offline script ( $0.60 \leq R_{\text{axis}} \leq 1.00$ ) and additionally ranks multiple candidate detections using a composite score that balances detection quality with temporal consistency:

$$\text{score} = \begin{cases} c \cdot A & \text{if no previous centre is known} \\ \frac{c \cdot A}{1 + d / d_{\text{max}}} & \text{otherwise} \end{cases} \quad (8.1)$$

where:

- $c \in [0, 1]$  is the **detection confidence** returned by the YOLO model (how certain the network is that it has found the pupil).
- $A = \frac{\pi}{4} w_{\text{box}} h_{\text{box}}$  is the **estimated ellipse area** in pixels squared.

- $d$  is the **Euclidean distance** in pixels between the current candidate's centre and the *previous frame's* detected pupil centre. A small  $d$  means the detection is spatially close to where the pupil was last seen.
- $d_{\max} = 160$  px is the **maximum expected inter-frame jump** — the largest realistic pupil displacement between two consecutive frames.

On the first frame, the score reduces to  $c \cdot A$ , simply preferring large confident detections. On subsequent frames, the denominator  $1 + d/d_{\max}$  penalizes detections that are far from the last known pupil position, providing **temporal stability** by suppressing spurious detections on the opposite side of the frame.

## 8.4 Optional ROI Masking

When enabled (`masking_enabled = True`), the detector crops the input frame to a **Region of Interest (ROI)** centered on the previously detected pupil before running YOLO inference. Searching a small crop is much faster than searching the entire frame, and also reduces false positives from other dark regions in the eye such as eyelashes or the iris boundary.

The crop boundaries are computed as:

$$x_1 = \max(0, x_c - r), \quad x_2 = \min(W, x_c + r) \quad (8.2)$$

$$y_1 = \max(0, y_c - r), \quad y_2 = \min(H, y_c + r) \quad (8.3)$$

where:

- $(x_c, y_c)$  is the **centre of the pupil detected in the previous frame** ( $x_c$  = horizontal pixel coordinate,  $y_c$  = vertical pixel coordinate).
- $r$  is the **crop radius** set to twice the detected major axis ( $r = 2 \times MA$ ), providing enough margin to accommodate natural pupil movement between frames.
- $W$  and  $H$  are the **full frame width and height** in pixels.
- The  $\max(0, \cdot)$  and  $\min(W, \cdot)$  clamps ensure the crop stays entirely within the image boundaries even if the pupil is near the frame edge.

The YOLO model runs on this smaller crop. Detection coordinates, which are expressed relative to the crop's top-left corner  $(x_1, y_1)$ , are then **offset back** to the full-frame coordinate system by adding  $(x_1, y_1)$  before drawing the overlay and computing the pupil area.

## 8.5 Recording and Data Logging

The live session flow operates as follows:

1. On launch, the camera stream and YOLO inference start immediately in the background worker thread.
2. The user presses **Record** to begin a timed 12-second recording session. This also triggers the LED light stimulus sequence.

3. During recording, frame-level area and timestamp data are collected in memory and flushed to `area_raw.csv` in batches of 20 entries to minimize I/O overhead on the Raspberry Pi's SD card.
4. After 12 seconds (or if the user presses **Stop**), the recording ends, the camera is released, and all captured raw frames are saved as `recorded_raw.mp4`.
5. The GUI then renders the area-vs-time plot and displays the fatigue classification result directly in the application window.

# Chapter 9

## Results and Discussion

### 9.1 Quantitative Evaluation

The performance of the YOLOv8-based system is evaluated using the same metrics as the previous semester: total frames, zero-area frames before and after applying the model, and the percentage reduction in zero-area frames.

Table 9.1: Zero-area frame counts before and after applying YOLOv8n (offline).

| Dataset | Total Frames | Zero-Area Before | Zero-Area After | Reduction (%) |
|---------|--------------|------------------|-----------------|---------------|
| D1      | 729          | 24               | 0               | 100.00%       |
| D2      | 4668         | 270              | 21              | 92.22%        |
| D3      | 8624         | 6211             | 340             | 94.53%        |

Table 9.2: Percentage of successfully detected frames before and after YOLOv8n (offline).

| Dataset | Detected Before (%) | Detected After (%) | Improvement |
|---------|---------------------|--------------------|-------------|
| D1      | 96.71%              | 100.00%            | +2.47%      |
| D2      | 94.22%              | 99.55%             | +5.33%      |
| D3      | 27.97%              | 96.06%             | +68.09%     |

### 9.2 Comparison with U-Net

Table 9.3 summarizes the key differences between the U-Net and YOLOv8n approaches across several dimensions.

Table 9.3: Comparison of U-Net vs. YOLOv8n approaches.

| Dimension                 | U-Net                    | YOLOv8n                        |
|---------------------------|--------------------------|--------------------------------|
| Training epochs           | 50                       | 50                             |
| Image size (training)     | $640 \times 480$         | $640 \times 640$               |
| Rectangular training      | No                       | Yes ( <code>rect=True</code> ) |
| Training hardware         | GPU                      | GPU (CUDA)                     |
| Inference stages          | 2 (U-Net + classical)    | 1 (direct detection)           |
| Inference speed (Pi)      | Slow                     | Fast (NCNN backend)            |
| Output type               | Full reconstructed image | Bounding box $(x, y, w, h)$    |
| Axis ratio filter         | No                       | Yes (0.60–1.00)                |
| Occlusion handling        | Moderate                 | Strong                         |
| Area computation          | Via re-running classical | Directly from box dims         |
| Manual annotation         | None required            | None required                  |
| Live real-time deployment | Not feasible             | Feasible (NCNN on Pi)          |

### 9.3 Limitations

- **Bounding box approximation.** The axis-aligned bounding box overestimates the pupil area for tilted pupils (e.g., during lateral gaze).
- **Confidence and threshold sensitivity.** Both the detection confidence threshold ( $\tau = 0.25$ ) and the fatigue slope threshold may require re-tuning for new recording environments or subjects.
- **Training data scope.** The training dataset is derived from a limited number of subjects and a single recording setup. Generalization to unseen subjects or hardware may degrade performance.
- **Inherited label noise.** Labels generated from the classical detector propagate any inaccuracies in classical ellipse fitting into the YOLO training set.
- **Threshold inconsistency.** The fatigue slope threshold differs between the offline script ( $-500 \text{ px}^2/\text{s}$ ) and the live GUI ( $-400 \text{ px}^2/\text{s}$ ). A unified, rigorously validated threshold is needed for reliable cross-mode comparison.

# Chapter 10

## Conclusion and Future Work

### 10.1 Conclusion

This report presented the 6th Semester continuation of the robust pupil detection project. Building on the hybrid classical + U-Net system developed in the 5th Semester, the primary contribution of this phase was the replacement of the U-Net image reconstruction model with a YOLOv8n object detector trained using 50 epochs,  $640 \times 640$  rectangular training, CUDA acceleration, and transfer learning from COCO-pretrained weights.

By contrast with the U-Net, YOLOv8n:

- Performs single-stage inference, directly outputting the pupil bounding box without requiring a secondary classical re-run.
- Achieves better robustness under eyelash occlusion and corneal reflections due to its region-based reasoning rather than contour-level analysis.
- Supports multiple deployment formats (PyTorch, NCNN, TFLite), with NCNN providing the best inference speed on the Raspberry Pi's ARM CPU.
- Enables a live real-time detection and fatigue analysis application, complete with a Reflex Index metric for continuous assessment of pupil light reflex strength.

The dataset for YOLOv8 training was constructed entirely automatically from the classical pipeline's successful detections, requiring no manual annotation. The trained model achieved near-perfect validation metrics ( $\text{mAP@0.5:0.95} = 0.9949$ ,  $\text{Recall} = 1.00$ ,  $\text{Precision} \approx 1.00$ ), with an F1 score that remains at its maximum over a wide range of confidence thresholds, demonstrating robust and reliable deployment characteristics.

### 10.2 Future Work

- **OBB (Oriented Bounding Box) detection.** YOLOv8 supports oriented bounding boxes, which would provide a better geometric fit for tilted pupils and reduce area estimation error compared to axis-aligned boxes.
- **Instance segmentation.** Using the `yolov8n-seg` variant to predict a pixel-level pupil mask would enable direct, high-accuracy area computation without the ellipse approximation.
- **Unified fatigue threshold.** A calibration study across multiple subjects and recording conditions should be conducted to establish a single validated fatigue slope threshold for both offline and live modes.

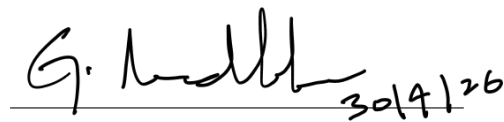
- **Multi-subject generalization.** Training on data from more subjects and under varied lighting conditions to improve model generalizability.
- **End-to-end fatigue classifier.** Replacing the heuristic slope threshold with a learned classifier operating on full pupil dynamics features such as reflex latency, constriction amplitude, and re-dilation speed.

# Declaration

I hereby declare that the project report entitled “**Robust Pupil Detection Using Classical Computer Vision and Deep Learning: Part II – Transition to YOLOv8-Based Detection**” submitted by me in fulfillment of the requirements for the B.Tech Project (EP3205) in the Department of Physics, Indian Institute of Technology Hyderabad, is my original work carried out under the guidance of **Dr. Vandana Sharma**.

I further declare that this work has not been submitted, either in part or in full, to any other institute or university for the award of any degree, diploma, or certificate.

Student Signature:

Handwritten signature of G. Madhukar in black ink, with the date 30/4/26 written below it.

G Madhukar  
EP23BTECH11011

Supervisor Signature:

---

Dr. Vandana Sharma  
Department of Physics  
IIT Hyderabad

Date: 30/04/2026

Place: Indian Institute of Technology Hyderabad

# Acknowledgements

I would like to express my sincere gratitude to my supervisor, **Dr. Vandana Sharma**, for her continued guidance, encouragement, and for providing access to the computing infrastructure necessary for training the deep learning models.

I am deeply thankful to **Varun Bhaiyya** (OP24BTECH11003) for his consistent help, technical suggestions, and support throughout the implementation of the YOLOv8 pipeline, the dataset generation scripts, and the live detection GUI.

I also extend my gratitude to **Parishkrith Bhaiyya** (PH25RESCH12001) for his insightful discussions and contributions that helped shape the direction of this work.

Their support has been invaluable to the progress and quality of this project.

# Bibliography

- [1] O. Ronneberger, P. Fischer, and T. Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, MICCAI, 2015.
- [2] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, *Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising*, IEEE Transactions on Image Processing, 2017.
- [3] B. Hassan, R. Ahmed, T. Hassan, and N. Werghi, *SIP-SegNet: A Deep Convolutional Encoder–Decoder Network for Joint Semantic Segmentation and Extraction of Sclera, Iris, and Pupil*, arXiv:2003.00825, 2020. <https://arxiv.org/abs/2003.00825>
- [4] A. Kalamatianos et al., *Analysis of pupil dynamics for neurological assessment: A review*, Frontiers in Human Neuroscience, 2022. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8973187/>
- [5] T. Santini, W. Fuhl, and E. Kasneci, *PuRe: Robust Pupil Detection for Real-Time Pervasive Eye Tracking*, Computer Vision and Image Understanding, 2018. <https://www.hci.uni-tuebingen.de/assets/pdf/publications/TWE052018.pdf>
- [6] A. Fuhl et al., *A Deep Learning Approach to Pupillometry*, SPIE Proceedings, Vol. 11843, 2021. <https://www.spiedigitallibrary.org/conference-proceedings-of-spie/11843/1184312/>
- [7] Ultralytics, *YOLOv8: A New State-of-the-Art Model for Object Detection*, <https://github.com/ultralytics/ultralytics>, 2023.
- [8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, *You Only Look Once: Unified, Real-Time Object Detection*, CVPR, 2016.

# List of Abbreviations

|             |                                                                                     |
|-------------|-------------------------------------------------------------------------------------|
| <b>YOLO</b> | You Only Look Once                                                                  |
| <b>CNN</b>  | Convolutional Neural Network                                                        |
| <b>CSP</b>  | Cross Stage Partial                                                                 |
| <b>C2f</b>  | Cross Stage Partial Bottleneck with 2 Convolutions                                  |
| <b>PAN</b>  | Path Aggregation Network                                                            |
| <b>FPN</b>  | Feature Pyramid Network                                                             |
| <b>DFL</b>  | Distribution Focal Loss                                                             |
| <b>NMS</b>  | Non-Maximum Suppression                                                             |
| <b>NCNN</b> | Neural network inference framework for ARM processors (no official expansion)       |
| <b>GUI</b>  | Graphical User Interface                                                            |
| <b>ROI</b>  | Region of Interest                                                                  |
| <b>IoU</b>  | Intersection over Union                                                             |
| <b>mAP</b>  | Mean Average Precision                                                              |
| <b>TP</b>   | True Positive                                                                       |
| <b>FP</b>   | False Positive                                                                      |
| <b>FN</b>   | False Negative                                                                      |
| <b>MA</b>   | Major Axis (the longer diameter of the fitted ellipse)                              |
| <b>ma</b>   | Minor Axis (the shorter diameter of the fitted ellipse)                             |
| $x_c$       | $x$ -coordinate of the centre of the detected pupil (horizontal position in pixels) |
| $y_c$       | $y$ -coordinate of the centre of the detected pupil (vertical position in pixels)   |
| <b>RI</b>   | Reflex Index                                                                        |
| <b>fps</b>  | Frames Per Second                                                                   |
| <b>GPU</b>  | Graphics Processing Unit                                                            |
| <b>CPU</b>  | Central Processing Unit                                                             |
| <b>CUDA</b> | Compute Unified Device Architecture (NVIDIA GPU parallel computing platform)        |
| <b>COCO</b> | Common Objects in Context (large-scale image dataset used for pretraining)          |
| <b>OBB</b>  | Oriented Bounding Box                                                               |